中間発表に向けたロードマップ

ブラウザ上での LLM 推論（WebLLM / WebGPU）と数値計算（Wasm / WebGPU）の実行性能に関する基礎検討

前提：中間発表の日付が未定のため、スケジュールは「第 1 週〜第 8 週（第 8 週＝中間発表）」の相対的な週で示しています。日程が決まり次第、各週に日付を当てはめてください。研究の問い（RQ）は暫定案で、指導教員との相談で確定させる想定です。

1. 研究の目的と問い（暫定）

ブラウザだけで LLM を動かす技術（WebLLM）が実用段階に入りつつある。一方、ブラウザ内の数値計算には CPU で動く WebAssembly（Wasm）と GPU を使う WebGPU の 2 つの選択肢がある。本研究では、まずブラウザ上での両者の性能特性を把握し、次に LLM の推論と行列計算が同じ GPU を共有したときの影響を明らかにする。

> **RQ1：**ブラウザ上の行列計算において、WebGPU（GPU）と Wasm（CPU）は、どの規模・条件でどちらが速いか。計算精度にはどのような差があるか。

> **RQ2：**LLM の推論（自然言語処理のための行列計算）と、ユーザーが依頼する行列計算（「A と B の積を計算して」「A の固有値を求めて」など）を同じ GPU で同時に実行すると、それぞれを単体で実行したときと比べて性能はどれだけ変化するか。

> **RQ3（調査）：**モデル自体を改良せずに性能を上げる手法（RAG など）や、高性能モデルを WebGPU で動かす取り組みについて、既存研究はどこまで進んでいるか。

2. これまでの成果

項目	内容
完了 Java → Wasm 変換	数値計算ライブラリ nfc（Java）を TeaVM で WasmGC に変換し、JS から呼び出せる手書きのブリッジを作成（固有値・LU 分解・特異値分解・FFT・行列指数関数）。
完了 API の自動抽出	JavaParser で nfc の全 417 ファイルを解析し、public な API（クラス・メソッド・引数・戻り値）を JSON に出力するツールを作成。
完了 ブリッジの自動生成	Python + jinja2 で、JSON から Java のエンドポイント・ブリッジと JS ラッパーを自動生成（69 メソッド）。手書き版と計算結果が一致すること、Node.js 上で Wasm として動作することを確認。
完了 資料	解析ツールと生成器の 1 行ずつの解説資料、Wasm 化の手順書（WASM_EXPORT_GUIDE.md）、nfc のクラス概要（NFC_CLASS_OVERVIEW.md）。

※ nfc の未対応機能（乱数・多項式・常微分方程式など）への対応と生成器の汎用化は、中間発表後の課題とする（5 章）。

3. 中間発表までの全体像

A. 先行研究調査
全期間で並行

→

B. 測定基盤づくり
WebLLM デモ・測定ページ

→

C. 基礎実験 1
WebGPU vs Wasm（RQ1）

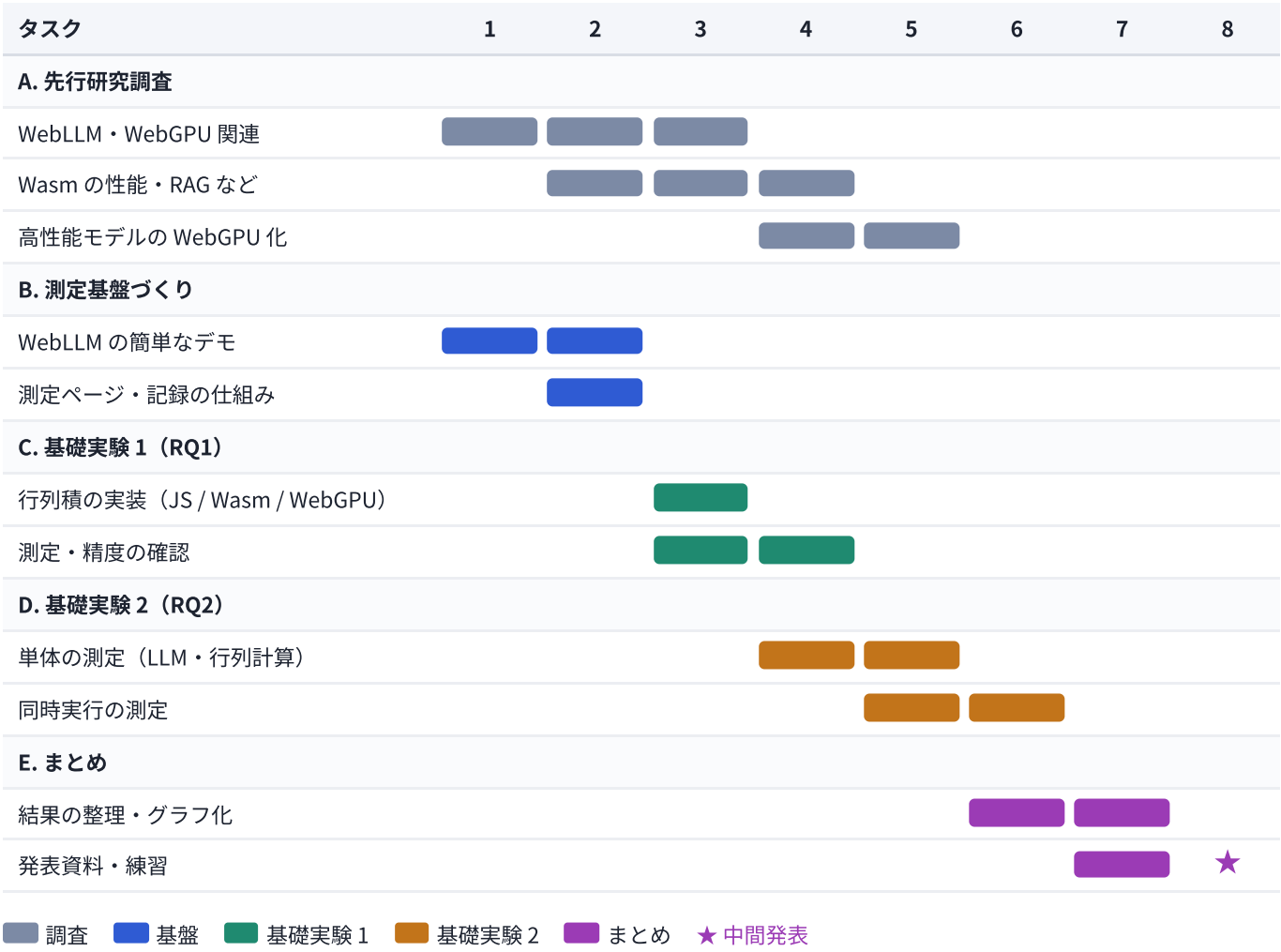
→

D. 基礎実験 2
同じ GPU での競合（RQ2）

→

E. まとめ
発表資料

スケジュール（第 8 週＝中間発表）



週ごとの到達目標（チェックポイント）

週	終わった時点で出来ていること
第 2 週	WebLLM で小さなモデルが動き、prefill / decode の速度を取得できる。測定条件（機材・ブラウザ）を記録する仕組みができている。調査の候補論文リストがある。
第 4 週	基礎実験 1 のグラフ（行列サイズ × 実行時間、精度の差）が 1 枚できている。先行研究の要約がある。
第 6 週	基礎実験 2 の 3 条件（LLM 単体・行列計算単体・同時実行）の測定が終わっている。
第 7 週	結果と考察がまとまり、発表資料の初稿ができている。指導教員に確認してもらう。
第 8 週	中間発表。

4. 各タスクの内容

A. 先行研究調査（RQ3）

テーマ	調べること・候補
ブラウザでの LLM 推論	WebLLM の論文（MLC チーム、2024 年 arXiv）、MLC-LLM / Apache TVM、Transformers.js（ONNX Runtime Web の WebGPU 版）
Wasm の性能	Wasm とネイティブ実行の比較（Jangda ら, "Not So Fast", USENIX ATC 2019）、Wasm の SIMD・マルチスレッドの効果
WebGPU の性能	WebGPU での汎用計算（GPGPU）のベンチマーク、WebGL との比較
モデルを改良せずに性能を上げる手法	RAG（Lewis ら, 2020）、ツール利用（Toolformer、PAL、ReAct）
高性能モデルの WebGPU 化	WebLLM が対応しているモデルの範囲、量子化とブラウザのメモリ上限、既存の移植事例。「動くようにする」こと自体はすでに実装例が多いため、新しさの出し方を検討する。

※ 書誌情報は調査時に原典で確認する。

B. 測定基盤づくり

- **WebLLM の簡単なデモ：**小さなモデルでチャットができるページを作り、prefill（プロンプトを読み込む段階）と decode（トークンを生成する段階）の速度を、WebLLM が出す統計値から取得できるようにする。
- **測定の作法をそろえる：**ウォームアップ後に複数回（例：10 回以上）測り、中央値とばらつきを記録する。GPU は「計算の完了を待ってから」時間を止める。
- **環境の記録：**CPU・GPU の型番、メモリ量、OS、ブラウザとそのバージョンを、測定結果と一緒に自動で保存する。
- **注意：**高精度のタイマーとマルチスレッドの Wasm は、ページの cross-origin isolation（特定の HTTP ヘッダの設定）を有効にしないと制限される。

C. 基礎実験 1：WebGPU と Wasm の比較（RQ1）

項目	内容
計算内容	行列積（全実装で同じアルゴリズムを用意しやすい）。余裕があれば固有値計算（Wasm 側は nfc の既存成果を使える。GPU 側は実装が難しいため後回し）。
比較対象	素の JS（基準）、Wasm（TeaVM で変換した nfc／C や Rust で書いたもの）、WebGPU（計算シェーダ）。TeaVM 版と C/Rust 版を両方測ると、遅さの原因が Wasm 自体か TeaVM かを切り分けられる。
変える条件	行列サイズ（例：16〜2048）。小さいと CPU、大きいと GPU が有利になる「逆転点」を探す。
測る指標	実行時間。GPU は「データ転送・シェーダのコンパイル・初回実行を含む／含まない」の両方を測る。
精度	WebGPU は基本的に単精度（f32）、Wasm・nfc は倍精度（f64）。同じ入力での誤差も比べる。

D. 基礎実験 2：同じ GPU での LLM 推論と行列計算（RQ2）

条件	測るもの
① LLM のみ	prefill / decode の速度（tokens/s）、出力トークン数
② 行列計算のみ	WebGPU での行列計算の実行時間（基礎実験 1 の結果を使う）
③ 同時に実行	LLM が文章を生成している間に行列計算を繰り返し実行し、①・②からどれだけ遅くなるかを測る

- **「同時」の作り方を明確にする：**「LLM が依頼文を読む → GPU で計算する」と順番に動く場合、実際にはほとんど重ならない。重なりの影響を見るには、意図的に同時に実行する状況を作る。順番に実行する場合の合計時間も参考として測る。
- **LLM に計算の手前で止めさせる条件：**プロンプトで「行列の読み取りまでで止めて」と指示した場合、出力は短くなるが、モデル内部の計算量そのものは変わらない（出力トークン数が減るだけ）。比べるときは出力トークン数も記録する。
- **GPU メモリ：**LLM はモデルの重みだけで数 GB を使うため、大きな行列ではメモリ不足で失敗する可能性がある。どのサイズで限界が来るかも結果として記録する。
- **実行場所：**行列計算を同じタブで動かすか Web Worker で動かすかで結果が変わりうるので、条件として記録する。

5. 中間発表以降の課題

課題	内容
後 nfc の未対応機能への対応	乱数・多項式・行列演算など。Java のオブジェクトを JS に番号（ハンドル）で渡す仕組みや、JS の関数を Java に渡す仕組み（常微分方程式など）が必要。
後 生成器の汎用化	入口クラスの選び方・型の変換表・名前の上書きを YAML の設定ファイルに出し、nfc 以外の Java ライブラリにも使えるようにする。
後 高性能モデルの WebGPU 化	先行研究調査の結果を見て、新しさが出せる切り口があれば着手する。

6. リスクと対策

リスク	対策
WebGPU で倍精度が使えない	精度の差そのものを結果として示す。必要なら倍精度を模擬する方法の文献を調べる。
GPU メモリ不足で同時実行が失敗する	小さいモデル・量子化モデルを使う。失敗するサイズも結果として記録する。
測定値のばらつきが大きい	回数を増やし、中央値と四分位範囲で示す。他のアプリを閉じ、電源設定を固定する。
機材・ブラウザで結果が変わる	測定環境を必ず記録する。可能なら 2 台以上の機材で確認する。
モデルのダウンロードに時間がかかる	初回にキャッシュさせ、測定にはダウンロード時間を含めない。

スケジュールの遅れ	優先順位は C（基礎実験 1）＞ D ＞ 固有値計算などの追加項目。D が間に合わない場合は、①②の単体測定と③の予備結果までを発表する。

7. 中間発表で示すもの（目標）

1. 研究の目的と RQ
2. これまでの成果（Java → Wasm の自動生成の仕組みと検証結果）
3. 基礎実験 1 のグラフ：行列サイズ × 実行時間（JS / Wasm / WebGPU）、精度の比較
4. 基礎実験 2 のグラフ：単体と同時実行での性能の変化
5. 先行研究の整理と、本研究の位置づけ
6. 今後の計画（5 章）

8. 指導教員と相談したい点

- RQ の範囲：RQ1・RQ2 を中心に据えてよいか。
- nfc の Wasm 化と自動生成を、研究の成果としてどこまで含めるか。
- 基礎実験 2 で想定する利用場面（LLM と行列計算を本当に同時に実行する場面があるか）。
- 測定に使う機材（研究室の PC、GPU の種類）。